

Chapter 7: Vector Database Embeddings with Pinecone

Lesson 1: Introduction to Pinecone

What is an Index?

- Store vectors
- Serve queries and other vector manipulations
- Each index contains records for each vector, including metadata
- Can create multiple indexes

Index Types: Pod-Based vs Serverless

Pod-Based:

- Choose hardware to create the index (pods)
- Pod type determines storage, query latency, and query throughput

Serverless (used in this course):

- No resource management needed
- Indexes scale automatically
- Run on cloud and store in blob
- Easier to use and often lower cost

Creating a Pinecone API Key

1. Create a Pinecone Starter account at pinecone.io
2. Head to "API Keys"
3. Copy your API key

Creating a Serverless Index

```
from pinecone import Pinecone, ServerlessSpec
```

```
pc = Pinecone(api_key="API_KEY")
```

```
pc.create_index(  
    name='datacamp-index',  
    dimension=1536,  
    spec=ServerlessSpec(cloud='aws', region='us-east-1')  
)
```

Checking Indexes

```
pc.list_indexes()
```

Returns a dict with index info: dimension, host, metric, name, spec, and status.

MANAGING INDEXES

Connecting to an Index

```
index = pc.Index('datacamp-index')
```

If the index doesn't exist, a 404 NotFoundException is raised.

Index Statistics

```
index.describe_index_stats()
```

Returns: dimension, index_fullness, namespaces, total_vector_count

Namespaces

Containers for partitioning indexes. Useful for:

- Separating datasets
- Data versioning
- Separating user groups

Note: This course focuses on the single namespace case.

Organizations

Hierarchy: Organization > Projects > Indexes > Namespaces

- Organization Owner: Permissions across entire org; manages billing, users, all projects
- Organization User: Restricted permissions; invited to specific projects; becomes owner of those projects

Deleting Indexes

```
pc.delete_index('datacamp-index')
```

VECTOR INGESTION

Vector Structure

Each vector is a dict with:

- id: unique string identifier
- values: list of floats (must match index dimension)
- metadata: optional dict of key-value pairs

Checking Dimensionality

```
vector_dims = [len(v['values']) == 1536 for v in vectors]
all(vector_dims) # Should be True
```

A dimension mismatch causes a 400 PineconeApiException.

Upserting Vectors

.upsert() means update or insert — adds new vectors or overwrites existing ones with the same ID.

```
index.upsert(vectors=vectors)
```

Vectors with Metadata

Metadata is extra data attached to each vector. Used for metadata filtering (Chapter 2).

```
{"id": "0", "values": [...], "metadata": {"genre": "productivity", "year": 2020}}
```

Lesson 2: Pinecone Manipulation with Python

Fetching vectors

Retrieve vectors based on their IDs

```
index.fetch(
    ids=['0', '1']
)

{'namespace': '',
 'usage': {'read_units': 1},
 'vectors': {'0': {'id': '0',
                   'metadata': {"genre": "productivity", "year": 2020},
                   'values': [0.025525547564029694, ...]},
             '1': {'id': '1',
                   'metadata': {"genre": "action", "year": 2023},
                   'values': [-0.0131468913, ...]}}
}
```

Index Query

- **.query() Method:** This method retrieves vectors similar to a given input vector using embeddings. Key parameters include:
 - **vector:** The input query vector.
 - **top_k:** The number of top matches to retrieve.

- `filter`: Additional filtering based on metadata.
 - `namespace`: The namespace for the search.
- **.fetch() Method**: This method retrieves vectors using unique identifiers (IDs) without utilizing embeddings. Key parameters include:
 - `ids`: A list of unique identifiers for the vectors.
 - `namespace`: The namespace from which to fetch the vectors.

Example Code:

Initialize the Pinecone client using your API key

```
pc = Pinecone(api_key="<PINECONE_TOKEN>")
```

```
index = pc.Index('datacamp-index')
```

Retrieve the MOST similar vector with the year 2024

```
query_result = index.query(
```

```
    vector=vector,
```

```
    top_k=1,
```

```
    filter={"year": 2024}
```

```
)
```

```
print(query_result)
```

Metadata Filtering

- **Updating Vectors**: You used the `.update()` method to modify existing vectors. This can involve changing the vector values or updating the metadata.

Example: To update a vector's metadata:

```
index.update(
```

```
    id="7",
```

```
    set_metadata={
```

```
        "genre": "thriller",
```

```
        "year": 2024
```

```
    }
```

```
)
```

- **Deleting Vectors**: You learned to remove vectors using the `.delete()` method, which helps in cleaning up the database and improving query performance.

Example: To delete vectors by ID:

```
index.delete(ids=["3", "4"])
```

- **Advanced Deletion:** You explored deleting vectors based on metadata filters and clearing entire namespaces, which can be useful for bulk deletions.

Updating and Deleting Vectors

Updating

```
index.update(
  id="1",
  values=[0.370695321, ...]
)
```

```
index.fetch(id=["1"])
```

Ensure:

list length = index dimensionality

Updating Vector Metadata

```
index.update(
  id="1",
  set_metadata={"genre": "comedy", "rating": 5}
)
```

Deleting vectors

```
index.delete(
  ids=["1", "2"]
)
```

Deleting vectors by-metadata

```
index.delete(
  filter={"genre": {"$eq": "action"}},
)
```

Deleting vectors within a namespace

```
index.delete(delete_all=True, namespace='namespace1')
```

Note

: Also removes the namespace!

Lesson 3: Performing Tuning with Ai Applications

Semantic Search With Pinecone

- **Initialization:** You configured Pinecone and OpenAI with your API keys and created a Pinecone index named "semantic-search-datacamp" with a dimensionality of 1536 and cosine similarity as the distance metric.
- **Document Preparation:** You prepared documents from the SQuAD dataset, which includes Wikipedia articles, and set up variables to streamline the indexing process.
- **Batch Processing:** You processed documents in batches, encoding them into embeddings using OpenAI's text-embedding-3-small model, and upserted these embeddings into the Pinecone index.
- **Semantic Search:** You performed a semantic search by encoding a query into embeddings and using Pinecone's `index.query()` method to find the most similar documents.

Here's a snippet demonstrating the creation of the Pinecone index:

```
# Initialize the Pinecone client using your API key
pc = Pinecone(api_key="<PINECONE_TOKEN>")

# Create Pinecone index
pc.create_index(
    name='semantic-search-datacamp',
    dimension=1536,
    metric='cosine',
    spec=PodSpec(environment="gcp-starter")
)

# Connect to index and print the index statistics
index = pc.Index('semantic-search-datacamp')
print(index.describe_index_stats())
```

- **Setting Up Pinecone and OpenAI:** You ensured both services were properly configured and connected to your existing Pinecone index.
- **Ingesting Data:** You ingested YouTube transcripts into the Pinecone index, encoding text into embeddings using the OpenAI API and upserting them into the index.
- **Retrieving Documents:** You implemented a retrieval function that fetches relevant documents based on a query by encoding the query and querying the Pinecone index.

- **Building Contextual Prompts:** You created a function to construct a prompt with retrieved documents and the user's query, which is then used to generate answers.
- **Generating Answers:** You developed a question-answering function using the OpenAI chat completion API to generate responses based on the contextual prompt.

Here's a snippet of the retrieval function:

```
def retrieve(query, top_k, namespace, emb_model):
    query_response = client.embeddings.create(input=query, model=emb_model)
    query_emb = query_response.data[0].embedding
    docs = index.query(vector=query_emb, top_k=top_k, namespace=namespace,
include_metadata=True)
    retrieved_docs = [doc['metadata']['text'] for doc in docs['matches']]
    sources = [(doc['metadata']['title'], doc['metadata']['url']) for doc in docs['matches']]
    return retrieved_docs, sources
```

Querying vectors cheat sheet (Pinecone + Python)

Core idea:

You store embeddings (vectors) in Pinecone, then send a *query vector* to find the most similar items.

1. Typical workflow

- Create query embedding

```
query_vector=embed_model("your query text")
```

- Send query to Pinecone
- Get back:
 - ids of matching vectors
 - scores (similarity)
 - metadata (original text, tags, etc.)

2. Key Python objects

- Client: connects to Pinecone
- Index: represents a specific collection of vectors
- Embedding model: turns text → vector

3. Important `index.query()` parameters

vector:

- The query embedding.

top_k:

- How many closest matches to return.

include_metadata:

- **True** to get stored text/labels.

include_values:

- **True** if you want the actual stored vectors back.

filter:

- Restrict results using metadata (e.g., only docs from a certain category).

4. Similarity scores

- Higher score → more similar.
- Metric depends on index type (often cosine or dot product).
- Don't overthink exact numbers—focus on ranking.

5. Best practices

- Same model for all embeddings (stored + query).
- Use metadata to narrow down irrelevant results.
- Store original text in metadata for easy display.
- Keep index dimensions consistent with your embedding model.

Step-by-step: querying vectors in Python

Step 1: Connect to Pinecone

python

```
from pinecone import Pinecone
```

```
pc = Pinecone(api_key="YOUR_API_KEY")
```

```
index = pc.Index("your-index-name")
```


What's happening:

You create a client (**pc**) and then grab a handle to your index (**index**).

Step 2: Create an embedding for your query text

python

```
query_text = "What is vector similarity?"
```

```
query_vector = embed_model.embed(query_text)
```

What's happening:

You turn natural language into a numeric vector that lives in the same space as your stored embeddings.

Step 3: Send the query to the index

python

```
results = index.query(
```

```
    vector=query_vector,
```

```
    top_k=5,
```

```
    include_metadata=True
```

```
)
```

What's happening:

Pinecone searches for the 5 most similar vectors and returns their ids, scores, and metadata.

Step 4: Inspect the results

python

```
for match in results["matches"]:
```

```
print("ID:", match["id"])

print("Score:", match["score"])

print("Text:", match["metadata"].get("text"))

print("-" * 40)
```

What's happening:

You loop through each match and read the stored info—this is what you'd show to a user or use as context.

Step 5: Add a metadata filter (optional)

```
python

results = index.query(

    vector=query_vector,

    top_k=5,

    include_metadata=True,

    filter={"category": {"$eq": "documentation"}}

)
```

What's happening:

You still do semantic search, but only among items whose **category** metadata equals **"documentation"**.